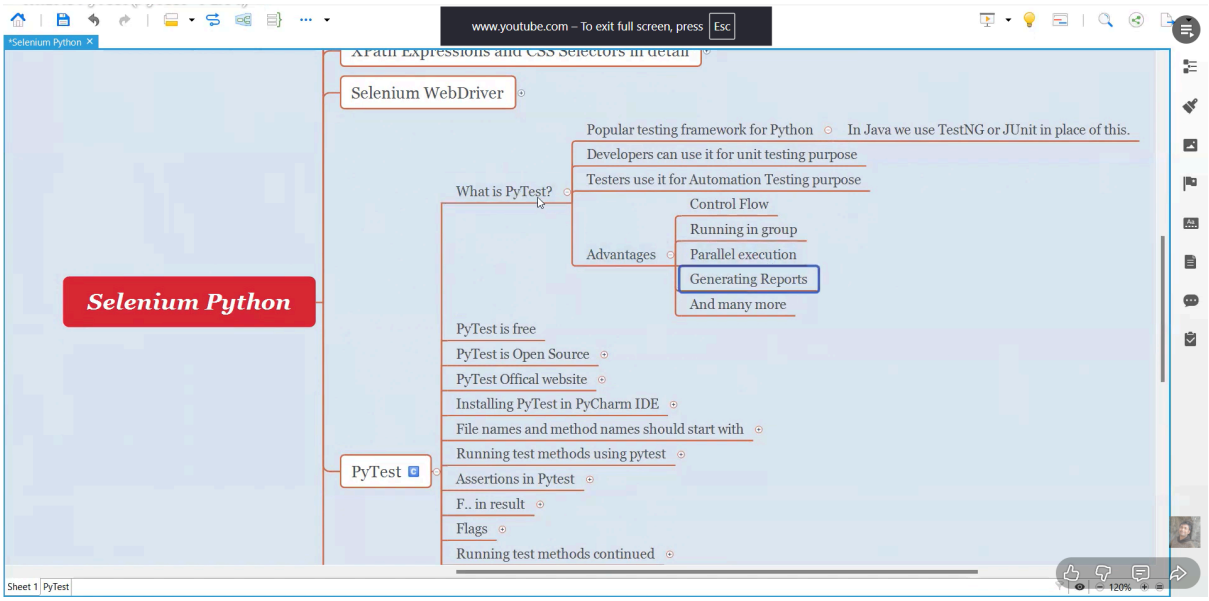
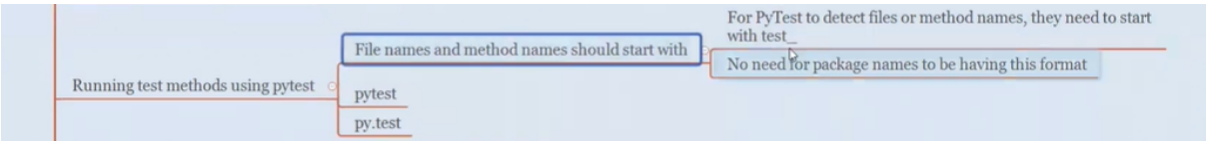
Pytest From Scratch

Course link: [LINK]

PyCharm Project Name: **Pytest_From_Scratch**



→ To run pytest files and methods we need to start each file name and method name with "**test**"



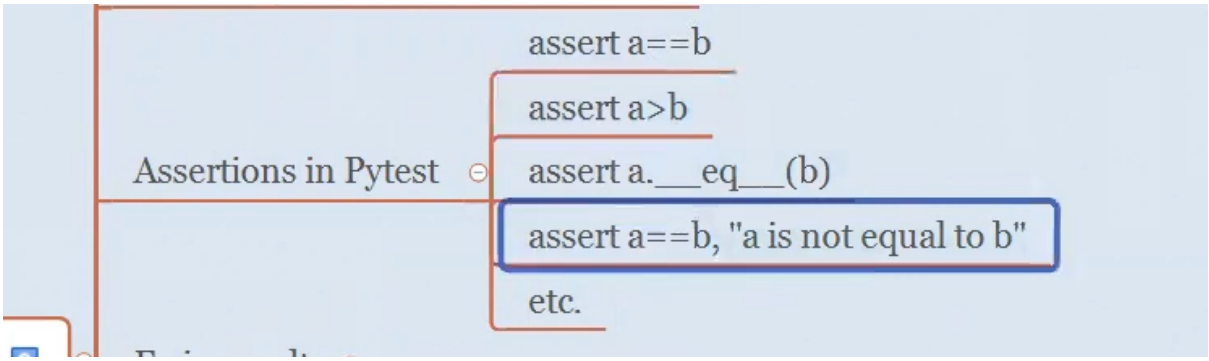
→ Refer to the test_SampleFile.py file in the samplefile project package of the project

If we want to run our tests, we need to give the commands

pytestt -r test_SampleFile

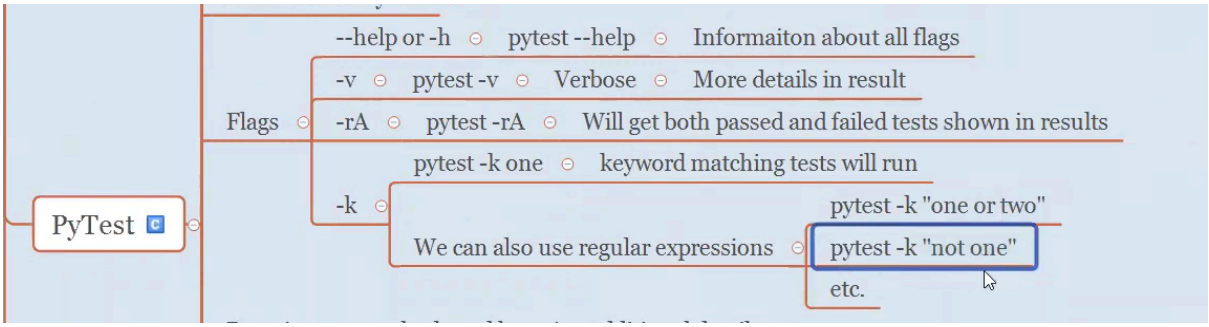
OR

py.test -r test_SampleFile



→ Refer to the test_assertions.py file in the samplefile project package of the project

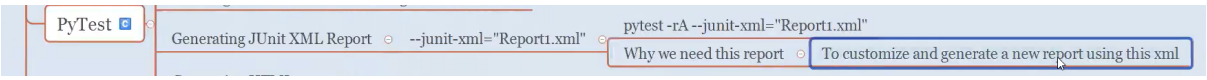
```
# In asserrtions if the test is passed then the message is not
displayed
# In asserrtions if the test is failed then the message is
displayed
```



In the same file test_assertions.py we can use the following flags to get details about the tests in the file by using the following files

- pytest -v → We will get more details for the result
- Pytest -rA → We will get results for both passed and failed tests
- Pytest -k “one” → If the keyword is matching in the particular function then it will execute.

→ **Generating JUnit XML Report:**



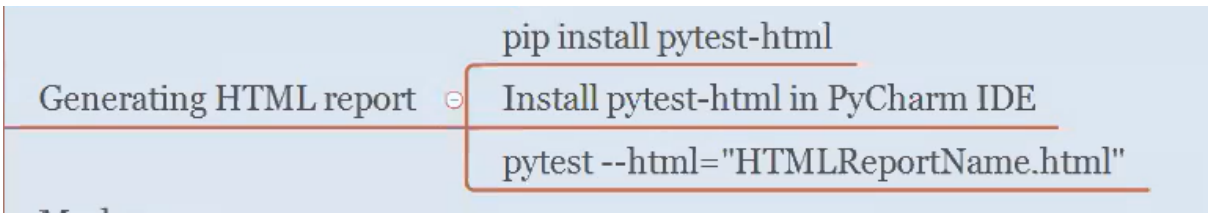
→ We can use the help command in pytest and identify the Junit command
pytest -h

To generate the XML file → pytest -rA --junit-xml="JUnitXMLReport.xml"

Refer to the xml file → JUnitXMLReport.xml in the project to find the xml report of failed testcases

What is the use of XML reports → we can use this report to generate customized reports, anyhow it is not gonna help us in real time.

→ **To generate html reports we can use the following command**
→ **pytest -rA --html="HTMLPytestReport.html"**



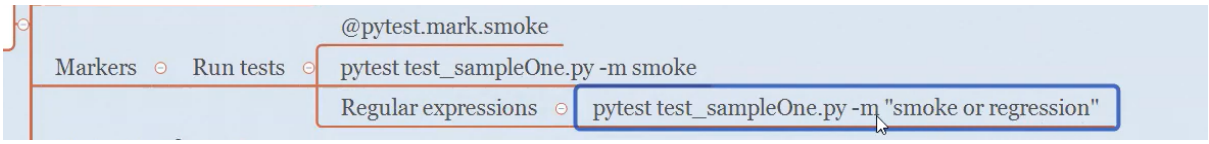
Markers

→ Markers in pytest are used to run a specific set of test cases based on the marker (keyword) that we give for that particular testcase.

→ Refer to the folder “Practice_markers” folder and verify that there are three Python files in the folder, and there are a set of test cases in every file which are marked with a specific set of markers like **“smoke”& “regression.”**

→ So to run the specific set of testcases based on the markers which we marked we need to use the command.

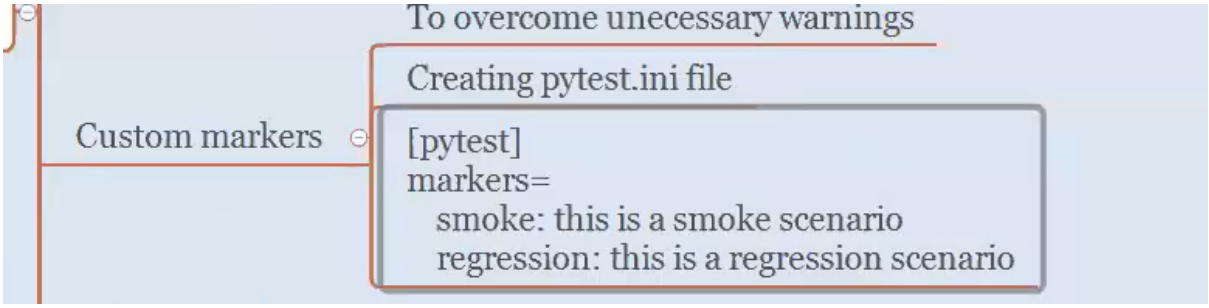
“pytest -rA -m smoke”, where it will run the testcases which are marked as **“smoke”.**



→ **Custom MARKERS**

→ Why do we need custom markers?

Ans. If our test case fails, we will get some unnecessary warnings in our terminal file so to overcome this we need to use the custom markers.



→ For that, we need to create a **"pytest.ini"** file in our project and define our own set of markers, then we will not get any errors.

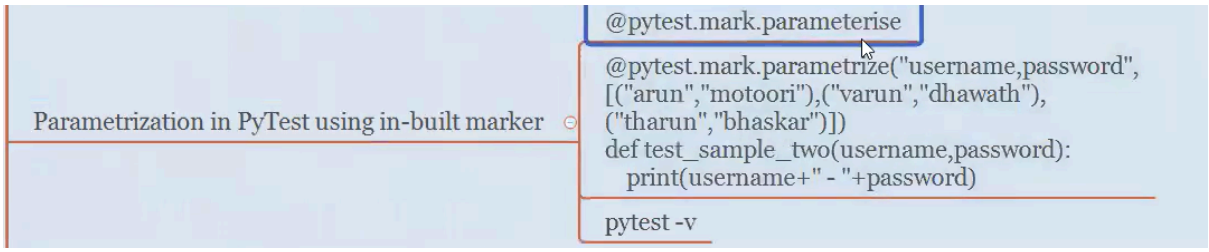
→ **To skip** a particular test case, we need to use the built-in marker **@pytest.mark.skip** Then the particular test will be skipped.

- Expected pass and expected fail category → refer to the Python file test_expectedfail

- **Parameterizing** in pytest

→ If we want to execute our tests with a different set of arguments, then we can use the parameters concept in markers.

→ Refer to the file **test_parameterMarker.py** file in the project



Fixtures

→ Fixtures are used to create the setup and teardown methods in pytest; in other words, if we want to run a Selenium test case, we need to do some setup to launch the webdriver and go to the webpage.

→ So, it's not possible to write the setup code for each test case, so here is where the **Fixture** comes into the picture.

→ In a fixture **yield** keyword is used to differentiate the setUp and tearDown methods, i.e log-into the browser followed by yield and Log-Out from the browser.

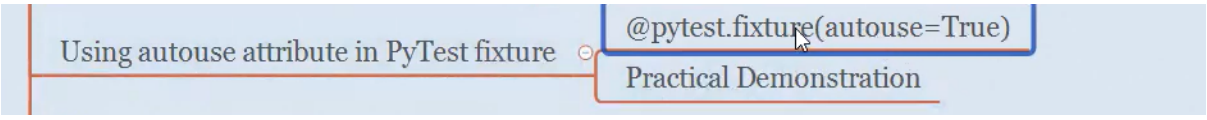
Refer to the **test_fixture01.py** file in the project.

What is the use of Conftest.py ?

→ We can consider it as a configuration file where all the configurations, like setting up the browser and quitting the browser, are placed in that file, and we are just importing the file into every other file if needed.

→ refer to the file conftest.py file in the project

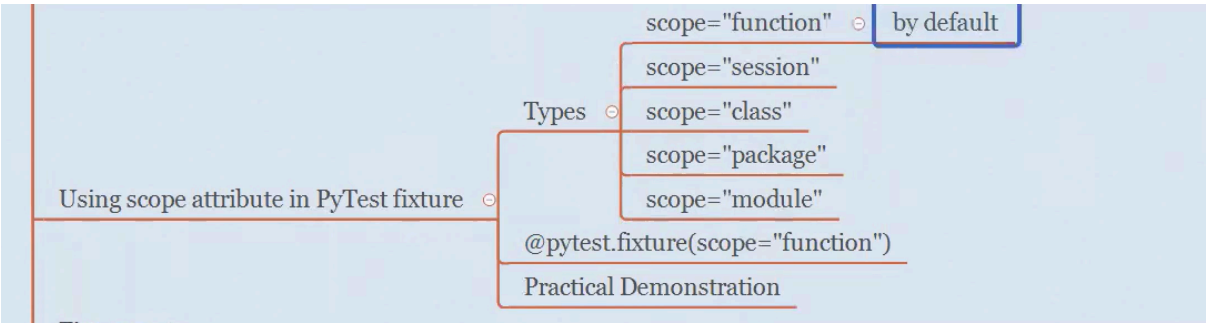
→ **What is the use of the Autouse attribute in fixture?**



→ We are creating fixtures to store the reusable code in our project, but if we want to use the fixture in any **test case,** we need to pass the fixture name as an argument.

→ If we forgot to give the fixture name as an argument, the test won't run, so to overcome this issue, we can use an **autouse=True** attribute, where the fixtures will run automatically, even without calling them in any function or test_case.

→ **What is the use of the Scope attribute in pytest?**



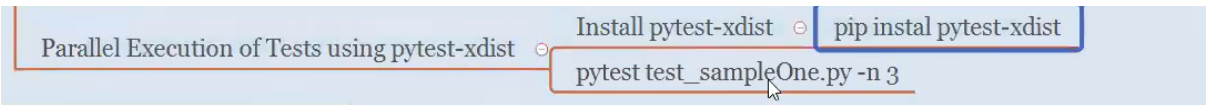
→ Basically, the fixture which we create runs the setUp and teatDown methods for every function, but we can also modify the behaviour of the fixture using the scope attribute.

→ For example, if we want to run the fixture only once per the **"Session",** i.e., we need to run the setup code only once before all the testcases and the testDown code after executing all the testcases, we can use the session option.

→ There are multiple sessions, and we can use them as per our requirement.

→ **How to run the tests parallely in pytest?**

→ Refer to the file **test_parallel.py** in the project



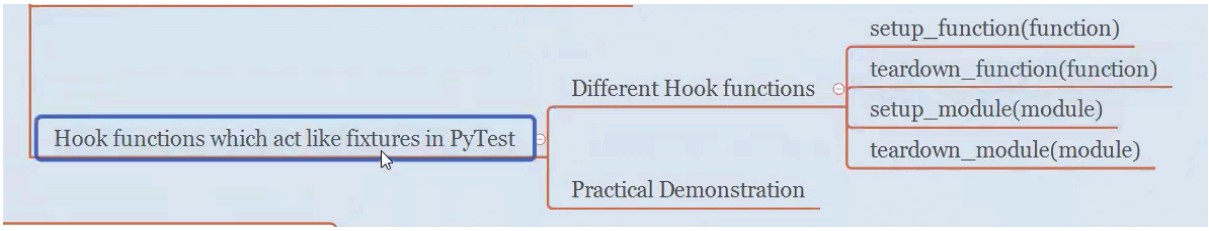
Instead of executing the test cases one by one in our python file we can run all the tests parallely to save our time, i.e no need to wait for the first test to run.

For that we need to install **pytest-xdist**

→ Use the code **pytest test_parallel.py -n4, which** means we can run the tests in parallel in 4 different browsers based on the number of tests.

Hook functions in pytest

→ Instead of **fixtures**, we can run Hook functions in pytest



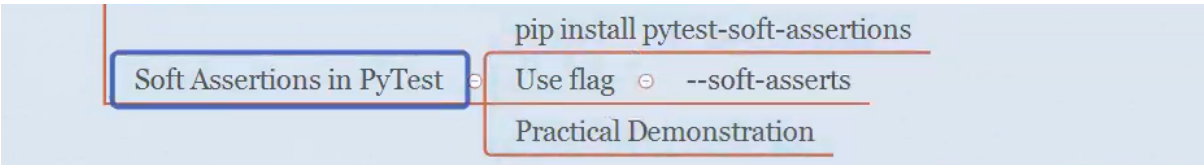
→ The setup and tear down functions are used to run before and after methods.

→ Refer the file **test_HookFunctions.py** file in the module.

Soft Assertions

⟶ **Hard Assertions:** If the assertion fails in our code, the execution will stop there only, and the code below will not be executed, but in soft assertions, it's not the case.

→ **Soft Assertions:** If there is an assertion in our code, the remaining code will be executed if we use soft assertions.



→ Refer to the file **test_SoftAssertions.py** in the project

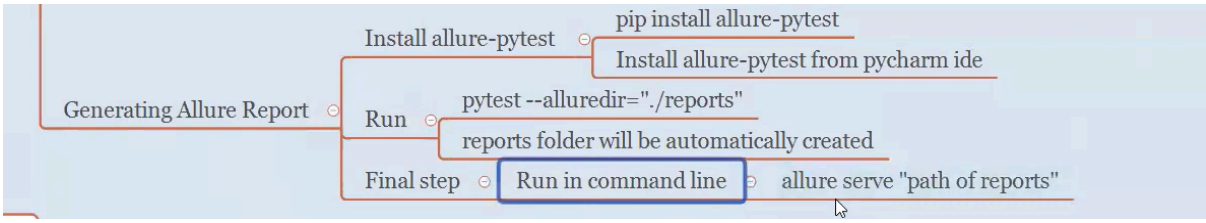
1. Install the soft-assertions package.
2. Run the command → **pytest -rA test_SoftAssertions.py --soft-asserts** , Then if there is any error we will get a tabel of data showing the errors in the terminal, but the other tests will also run without any issues.

```
    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.is_failed:
>           raise AssertionError(f'\n{self.draw_results()}')
E           AssertionError:
E           +------------------------------+-----------------+---------------+------------+
E           |          expression          |    condition    | current value |  expected  |
E           +==============================+=================+===============+============+
E           | assert driver.title.__eq__(ex | <Not condition> | False         | <Not None> |
E           | pected_title)                |                 |               |            |
E           +------------------------------+-----------------+---------------+------------+
```

→ Now we created a different project to integrate selenium with pytest, which is called

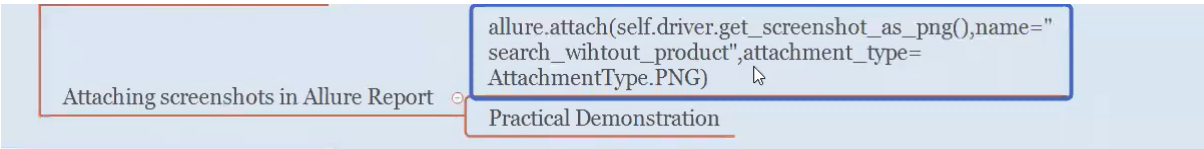
SeleniumPython_Pytest_Project

→ After creating all the testcases and executing them properly, we need to generate the reports of our passed or failed test cases.

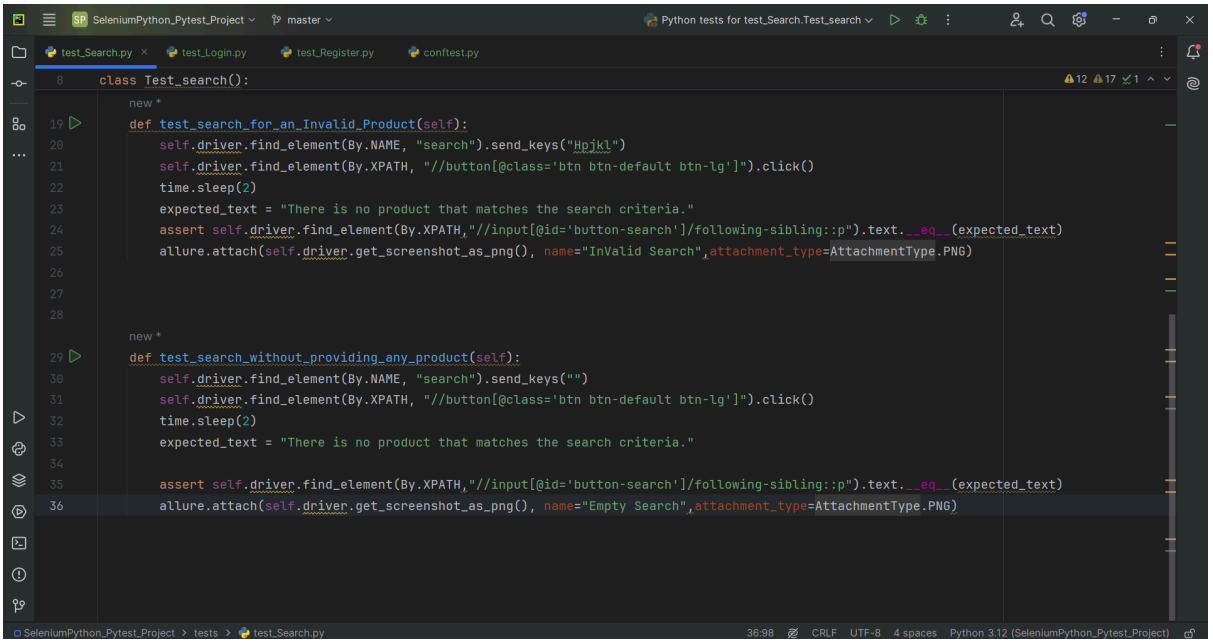


→ We need to use the **Allure package** to generate reports

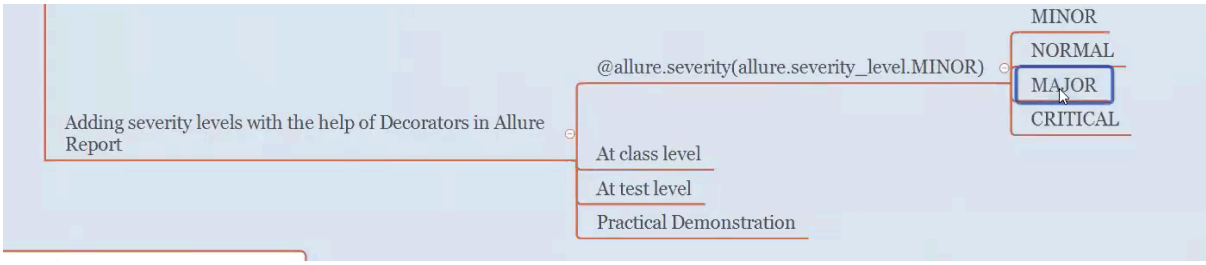
→ How to attach screenshots in allure



→ refer the fille **test_Search.py**



→ How to attach **Severity** to the test cases



→ Refer to the same file **test_Search.py**

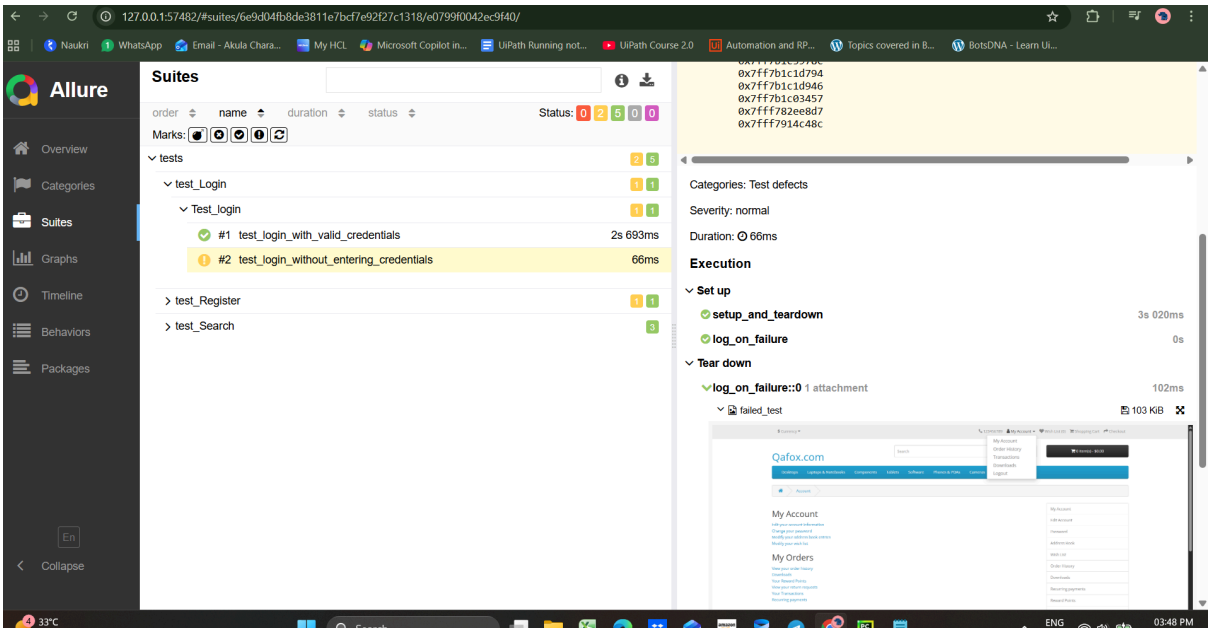
To take a screenshot on failure, use the following code in the conftest.py file :

```
#Code to take screenshot of the failure

@pytest.fixture()
def log_on_failure(request):
    yield
    item = request.node
    if item.rep_call.failed:
        allure.attach(driver.get_screenshot_as_png(),
name="failed_test",attachment_type=AttachmentType.PNG)


@pytest.hookimpl(hookwrapper=True, tryfirst=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()
    setattr(item, "rep_" + rep.when, rep)
    return rep
```

If there is any failure then it will show in the allure reports like this



→ **How to Parameterize Fixtures?**

Now the code will run all three browsers if our computer has all three different browsers' webdrivers

The code:

```
@pytest.fixture(params=['chrome','firefox','edge'])
def setup_and_teardown(request):
    global driver
    if request.param == "chrome":
        driver = webdriver.Chrome()
    '''elif request.param == "firefox":
        driver = webdriver.Firefox()
    elif request.param == "edge":
        driver = webdriver.Edge()'''
    driver.maximize_window()
    driver.get("https://tutorialsninja.com/demo/")
    request.cls.driver = driver
    yield
    driver.quit()
```

→ Refer to the github repository **SeleniumPython_Pytest_Project**

The End